MINI PROJECT REPORT

On

HPC-Based Parallel Machine Learning Training System

By

Anand More (BECOC34)

Under the guidance of

(Dr. Vijay kotkar)

Department of Computer Engineering

Pimpri Chinchwad College of Engineering and Research, RavetPlot No: B, Sector No. 110, gate no 1, Laxminagar, Ravet, Pune,Pin.: 412101

Savitribai Phule Pune University [2025-2026]

Department of Computer Engineering

Pimpri Chinchwad College of Engineering and Research,

Ravet

CERTIFICATE

This is certify that **Anand More** from Fourth Year Computer Engineering has successfully completed her mini project work titled "**HPC-Based Parallel Machine Learning Training System**" at Pimpri Chinchwad College of Engineering and Research, Ravet in the partial fulfillment of the Bachelor's Degree in Engineering.

Mrs. Shrinika Inamdar	Dr. Vijay A. Kotkar	Prof. Dr. Harish Tiwari
Guide	**Head of Department**	**Principal**

ABSTRACT

High Performance Computing (HPC) has become an essential tool for solving computationally intensive problems in modern applications. Machine Learning (ML) algorithms, especially when trained on large datasets, require significant computational resources and time.

This project presents a system that leverages HPC techniques to improve the performance of ML model training. It implements and compares four different approaches: serial execution, parallel processing, advanced parallel techniques, and distributed computing using Message Passing Interface (MPI).

The system integrates a full-stack architecture consisting of a React-based frontend, a Node.js backend, and Python-based ML modules. Experimental results show that parallel and distributed methods significantly reduce training time, with MPI providing the best performance.

Contents

1 **5**

1.1 Introduction 5

1.2 Literature Survey 5

2 **6**

2.1 Objective 6

2.2 System Architecture 6

2.3 Frontend 6

2.4 Backend 6

2.5 Machine Learning Layer 6

2.6 HPC Layer 6

2.7 Working Principle 7

3 Methodology **8**

3.1 Serial Execution 8

3.2 Parallel Execution 8

3.3 Advanced Parallel Execution 8

3.4 MPI-Based Distributed Execution 8

4 **9**

4.1 Implementation Details 9

4.2 Results 9

4.3 Discussion 9

5 **10**

5.1 Advantages 10

5.2 Limitations 10

5.3 Future Scope 10

5.4 Conclusion 10

Chapter 1

1.1 Introduction

Machine Learning has become a fundamental component of modern technology, powering applications such as recommendation systems, medical diagnosis, and smart agriculture. However, training ML models on large datasets is computationally expensive and time-consuming.

High Performance Computing (HPC) provides a solution by enabling parallel and distributed processing. By utilizing multiple cores or machines, HPC significantly reduces computation time and improves efficiency.

This project focuses on applying HPC techniques to ML training and comparing different approaches to understand their impact on performance.

1.2 Literature Survey

Several studies have explored the use of parallel computing in machine learning. Traditional ML algorithms often suffer from scalability issues when dealing with large datasets.

Research shows that:

- Parallel processing improves training speed by utilizing multi-core CPUs.
- Distributed systems using MPI allow tasks to be divided across multiple nodes.
- Libraries such as Joblib and frameworks like Hadoop and Spark are widely used for large-scale data processing.

This project builds upon these concepts and implements a practical system for performance comparison.

Chapter 2

2.1 Objective

- To design a system for ML training using HPC techniques
- To implement serial, parallel, and distributed training approaches
- To compare execution time and efficiency
- To demonstrate real-world application of MPI
- To visualize results using an interactive dashboard

2.2 System Architecture

The system consists of multiple layers:

2.3 Frontend

Developed using React.js, the frontend provides an interface to trigger different training methods and visualize results.

2.4 Backend

Built with Node.js and Express, the backend exposes APIs that execute Python scripts using child processes.

2.5 Machine Learning Layer

Handles dataset processing and model training using Python and Scikit-learn.

2.6 HPC Layer

Implements parallelism using Joblib and distributed computing using MPI.

2.7 Working Principle

The working of the system is as follows:

1. User selects a training method from the frontend.
2. A request is sent to the backend API.
3. The backend executes the corresponding Python script.
4. The ML model is trained using the selected method.
5. Execution time is measured and returned to the frontend.
6. Results are displayed along with graphical comparison.

Chapter 3

Methodology

3.1 Serial Execution

Processes the entire dataset sequentially using a single core. It is simple but inefficient for large datasets.

3.2 Parallel Execution

Uses multiple CPU cores to perform computations simultaneously, reducing execution time.

3.3 Advanced Parallel Execution

Divides the dataset into chunks and processes them independently, improving load distribution.

3.4 MPI-Based Distributed Execution

Splits the dataset across multiple processes. Each process trains independently, and results are aggregated by a master process.

Chapter 4

4.1 Implementation Details

- Dataset: Crop dataset
- Model: Random Forest Classifier
- Preprocessing: Label encoding
- Backend APIs: /train/serial, /train/parallel, /train/mpi
- Visualization: Bar chart using Chart.js

4.2 Results

Method	Time (seconds)
Serial	2.13
Parallel	1.20
Advanced	0.95
MPI	0.81

4.3 Discussion

The results demonstrate that:

- Parallel processing reduces computation time significantly
- Advanced techniques further improve efficiency
- MPI provides maximum speedup due to distributed execution

Chapter 5

5.1 Advantages

- High efficiency
- Reduced training time
- Scalable system
- Practical HPC implementation

5.2 Limitations

- Requires setup of MPI environment
- Limited to CPU-based execution
- Hardware-dependent performance

5.3 Future Scope

- GPU-based ML training
- Cloud deployment
- Big data integration
- Real-time analytics

5.4 Conclusion

This project demonstrates the effectiveness of HPC techniques in reducing ML training time. MPI-based distributed computing provides the best performance among all methods. The system integrates full-stack development with parallel computing, making it a practical solution for real-world applications.